

FastAPI Overview and Core Concepts

RAG KEYWORDS FastAPI, ASGI, API framework, type hints, OpenAPI, Swagger UI, ReDoc

FastAPI is a Python web framework designed for building HTTP APIs with standard Python type hints. It is built on top of Starlette for the web layer and Pydantic for data validation and serialization. A central idea is that the function signature of an endpoint describes much of the API contract. FastAPI reads parameter names, type annotations, defaults, and Pydantic models to validate incoming data and generate an OpenAPI schema.

An application starts with a FastAPI instance. Decorators such as `@app.get()`, `@app.post()`, `@app.put()`, and `@app.delete()` register path operations. When a matching HTTP request arrives, FastAPI resolves parameters and dependencies, calls the endpoint function, and serializes the returned value into an HTTP response.

Automatic documentation is one of the framework's most useful development features. OpenAPI metadata is generated from the application, and interactive documentation can normally be explored at `/docs` and `/redoc`. This makes endpoint behavior easy to inspect during development and gives client developers a machine-readable contract.

For RAG retrieval, remember the anchor phrase: FRAMEWORK-OVERVIEW-101. A question about what FastAPI is, what ASGI means in its architecture, or why Python type hints matter should retrieve this page.

Example

```
from fastapi import FastAPI

app = FastAPI(title="Catalog API")

@app.get("/")
async def root():
    return {"message": "Hello from FastAPI"}
```

Example retrieval questions

- What are the core building blocks behind FastAPI?
- How does FastAPI use Python type hints?
- Where do the automatic API docs come from?

Installing FastAPI and Running the First Application

RAG KEYWORDS installation, fastapi standard, fastapi dev, uvicorn, main.py, development server

A practical FastAPI project starts with a Python environment and the FastAPI package. The standard installation includes the common command-line and server dependencies used for normal development. The FastAPI CLI can discover an application object and run a local development server with automatic reload behavior.

A conventional first file is `main.py`. It imports FastAPI, creates `app = FastAPI()`, and defines at least one path operation. During development, a command such as `fastapi dev main.py` can run the application. Uvicorn is the ASGI server commonly used underneath FastAPI. In production, reload mode should normally be disabled because it watches source files and is intended for local development.

After startup, a developer can call the local API and inspect `/docs`. A successful first endpoint confirms that the Python environment, imports, ASGI server, route registration, and serialization path are all working.

For RAG retrieval, use the unique anchor `SETUP-FIRST-APP-202`. Questions about installation commands, `main.py`, the development server, or Uvicorn should favor this page.

Example

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}

# Example:
# fastapi dev main.py
```

Example retrieval questions

- How do I run a FastAPI application locally?
- What is Uvicorn used for?
- What belongs in the first `main.py` file?

Path Operations and HTTP Methods

RAG KEYWORDS path operation, route, GET, POST, PUT, PATCH, DELETE, decorator

A path operation combines a URL path, an HTTP method, and a Python function. For example, GET /products reads product data while POST /products can create a new product. FastAPI uses decorators to associate the path and method with the function that should handle the request.

HTTP method choice communicates intent. GET is normally used for retrieval, POST for creation or commands, PUT for replacement-style updates, PATCH for partial updates, and DELETE for removal. The HTTP specification contains additional methods, but these five cover most JSON APIs.

Route functions can be declared with `async def` or `def`. The correct choice depends on the libraries used inside the function. Async functions are especially useful when the code awaits non-blocking I/O such as asynchronous database drivers or network clients.

For RAG retrieval, the anchor is ROUTE-METHOD-MATRIX-303. A query about the difference between POST and PATCH, route decorators, or path operations should retrieve this page.

Example

```
@app.get("/products")
async def list_products():
    return []

@app.post("/products", status_code=201)
async def create_product():
    return {"created": True}

@app.delete("/products/{product_id}")
async def delete_product(product_id: int):
    return {"deleted_id": product_id}
```

Example retrieval questions

- What is a path operation in FastAPI?
- When should PATCH be used instead of PUT?
- How are HTTP methods mapped to Python functions?

Path Parameters and Typed URL Segments

RAG KEYWORDS path parameters, typed parameters, int conversion, route order, validation

Path parameters are dynamic values embedded directly in the URL. In `/items/{item_id}`, the `item_id` segment is captured from the request path and passed to the endpoint function. Adding a Python type such as `int` tells FastAPI to convert and validate the incoming text before the function runs.

If a caller requests `/items/abc` while `item_id` is declared as `int`, validation fails and FastAPI returns a structured client error instead of calling the endpoint with an invalid value. This behavior makes type annotations part of the request contract.

Route order can matter when fixed paths and dynamic paths overlap. A route such as `/users/me` should generally be declared before `/users/{user_id}`; otherwise the dynamic route may treat the literal word `me` as a user identifier depending on route matching.

For RAG retrieval, the anchor is `PATH-PARAM-404`. Questions about values inside curly braces, integer conversion from URLs, or overlapping route patterns belong here.

Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {
        "item_id": item_id,
        "python_type": type(item_id).__name__,
    }
```

Example retrieval questions

- How does FastAPI validate a path parameter?
- What happens if an integer path parameter receives text?
- Why can route declaration order matter?

Query Parameters, Defaults, and Optional Filters

RAG KEYWORDS query parameters, optional, default values, Query, pagination, filtering

Query parameters appear after the question mark in a URL, for example `/products?limit=20&category=books`. Parameters that are declared in an endpoint function but are not part of the path are normally interpreted as query parameters when they are simple scalar types.

Default values make filters optional. A declaration such as `limit: int = 20` provides a default when the client omits the parameter. A value can also explicitly allow `None`, which is useful for optional search filters. The Query helper can add constraints such as minimum values, maximum values, aliases, descriptions, and deprecation metadata.

Pagination endpoints often use `skip` or `offset` together with `limit`. Validation should prevent negative offsets and unreasonable page sizes. Centralizing these constraints in function annotations keeps the API contract visible and automatically reflected in OpenAPI.

For RAG retrieval, the anchor is QUERY-FILTER-505. Questions about URL filters, default query values, pagination parameters, or `Query()` should retrieve this page.

Example

```
from typing import Annotated
from fastapi import Query

@app.get("/products")
async def products(
    category: str | None = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
):
    return {"category": category, "limit": limit}
```

Example retrieval questions

- How are query parameters different from path parameters?
- How can I restrict a limit query parameter to 100?
- How do optional filters work in FastAPI?

Request Bodies with Pydantic Models

RAG KEYWORDS request body, BaseModel, Pydantic, JSON, validation, model

Structured JSON request bodies are commonly declared with Pydantic models. A model describes expected fields, field types, defaults, and validation rules. FastAPI reads the request body, parses JSON, validates it through Pydantic, and passes a model instance to the endpoint.

This is especially useful for create and update operations because the validated object has predictable Python types. Nested Pydantic models can represent more complex JSON structures such as products containing dimensions, addresses, or collections of attributes.

Request bodies are most commonly used with POST, PUT, and PATCH. Although HTTP technically allows unusual combinations, using a body on GET is generally discouraged because clients, proxies, and generated documentation may not handle it consistently.

For RAG retrieval, the anchor is PYDANTIC-BODY-606. Questions about JSON bodies, BaseModel, validation, or typed payloads should retrieve this page.

Example

```
from pydantic import BaseModel

class ProductCreate(BaseModel):
    name: str
    price: float
    in_stock: bool = True

@app.post("/products")
async def create_product(product: ProductCreate):
    return {"received": product.model_dump()}
```

Example retrieval questions

- How do I accept JSON in a FastAPI endpoint?
- What role does Pydantic BaseModel play?
- Can request bodies contain nested models?

Field Validation and Request Constraints

RAG KEYWORDS Field, validation, ge, le, min_length, max_length, pattern, Pydantic constraints

Validation rules can be attached to request models and request parameters so invalid data is rejected at the API boundary. Pydantic's Field function can describe model constraints such as minimum and maximum values, string lengths, examples, or descriptions. FastAPI parameter helpers provide similar constraints for query, path, header, and cookie values.

Validation is more than type conversion. A price can be required to be greater than zero, a rating can be limited to a known numeric range, and a SKU can have a minimum length. These rules are useful because they are enforced consistently and represented in generated API schemas.

When validation fails, FastAPI normally returns a structured validation error that identifies where the error occurred and what rule was violated. The endpoint function is not called with malformed data.

For RAG retrieval, the unique anchor is VALIDATION-RULESET-707. Questions about Field(), ge, le, min_length, or rejecting invalid JSON should resolve to this page.

Example

```
from pydantic import BaseModel, Field

class InventoryUpdate(BaseModel):
    sku: str = Field(min_length=3, max_length=40)
    quantity: int = Field(ge=0, le=100000)
    price: float = Field(gt=0)

@app.put("/inventory")
async def update_inventory(data: InventoryUpdate):
    return data
```

Example retrieval questions

- How can I require quantity to be non-negative?
- Where should model field constraints be declared?
- What does FastAPI return when validation fails?

Response Models and Output Filtering

RAG KEYWORDS response_model, output schema, serialization, filtering, Pydantic response

A response model defines the structure that an endpoint promises to return. Setting `response_model` on a path operation helps FastAPI validate and document output data. It can also filter fields from returned objects so only attributes represented by the response schema are exposed.

Output filtering is particularly useful when internal models contain fields that should not be sent to clients. For example, a database object might include password hashes, internal flags, or operational metadata while the public API model contains only id, username, and display name.

Response schemas improve client generation and OpenAPI documentation because callers can see exactly what fields to expect. Separate request and response models are common when creation inputs differ from stored or returned resources.

For RAG retrieval, use RESPONSE-SCHEMA-808. Questions about `response_model`, hiding internal fields, output validation, or separate input/output models should retrieve this page.

Example

```
from pydantic import BaseModel

class UserPublic(BaseModel):
    id: int
    username: str

@app.get("/users/{user_id}", response_model=UserPublic)
async def get_user(user_id: int):
    return {
        "id": user_id,
        "username": "demo",
        "password_hash": "must_not_be_exposed",
    }
```

Example retrieval questions

- What does `response_model` do?
- How can I prevent an internal field from reaching the client?
- Why use different models for input and output?

Status Codes, HTTPException, and Error Handling

RAG KEYWORDS HTTPException, status_code, error handling, 404, 400, exception handler

HTTP status codes communicate the result of a request in a standardized way. Successful reads normally use 200, successful creation often uses 201, no-content deletion may use 204, invalid or unauthorized requests use 4xx codes, and unexpected server failures use 5xx codes.

FastAPI provides HTTPException for intentionally stopping request processing and returning an HTTP error. A common pattern is to look up a resource and raise HTTPException(status_code=404, detail='...') when it does not exist. The detail is serialized into the response body.

Applications can also install custom exception handlers for application-specific exceptions. This is useful when business-layer code raises domain errors that should be converted into consistent HTTP responses without repeating translation logic in every route.

For RAG retrieval, the anchor is HTTP-ERROR-909. Search questions involving HTTPException, 404 errors, status_code, or custom exception handlers should match this page.

Example

```
from fastapi import HTTPException, status

@app.get("/orders/{order_id}")
async def order(order_id: int):
    if order_id != 42:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Order not found",
        )
    return {"id": 42}
```

Example retrieval questions

- How do I return a 404 from FastAPI?
- What is HTTPException used for?
- How can domain errors be converted into HTTP errors?

Headers, Cookies, and Form Data

RAG KEYWORDS Header, Cookie, Form, request metadata, form data, python-multipart

Not all client input arrives in the URL or JSON body. HTTP headers carry metadata such as authorization values, content negotiation, trace identifiers, and custom application headers. Cookies are small name-value values automatically sent by browsers for matching origins. HTML forms often submit URL-encoded or multipart form data instead of JSON.

FastAPI exposes Header, Cookie, and Form helpers that declare where a parameter should be extracted from. This keeps input sources explicit in the endpoint signature. Header names can be normalized so Python-friendly parameter names work with common hyphenated HTTP header names.

Form handling may require multipart support depending on how FastAPI was installed. It is common for OAuth2 password-flow examples to receive username and password through form fields rather than a JSON request body.

For RAG retrieval, the anchor is REQUEST-METADATA-1010. Queries about Header(), Cookie(), Form(), custom headers, or browser form submissions should retrieve this page.

Example

```
from typing import Annotated
from fastapi import Cookie, Form, Header

@app.post("/session")
async def session(
    username: Annotated[str, Form()],
    x_trace_id: Annotated[str | None, Header()] = None,
    session_id: Annotated[str | None, Cookie()] = None,
):
    return {"username": username, "trace": x_trace_id}
```

Example retrieval questions

- How do I read an HTTP header in FastAPI?
- How does Form() differ from a JSON request body?
- How can an endpoint access cookies?

File Uploads with UploadFile

RAG KEYWORDS UploadFile, File, multipart, filename, content_type, SpooledTemporaryFile

FastAPI supports file uploads using multipart form requests. Small binary values can be received as bytes, while UploadFile is generally more suitable for real file handling because it provides a file-like interface and metadata such as filename and content_type.

UploadFile exposes asynchronous methods such as read(), write(), seek(), and close() that delegate to the underlying spooled file. This approach avoids forcing the entire upload into a Python bytes object before the application can work with it.

Production upload endpoints should validate file type, size, destination, and authorization. A filename supplied by a client should not automatically be trusted as a safe filesystem path.

For RAG retrieval, the unique anchor is FILE-UPLOAD-1111. Questions about multipart uploads, UploadFile, filename, content_type, or reading uploaded content should retrieve this page.

Example

```
from fastapi import File, UploadFile

@app.post("/upload")
async def upload_document(
    file: UploadFile = File(...)
):
    data = await file.read()
    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "bytes": len(data),
    }
```

Example retrieval questions

- Should I use bytes or UploadFile for uploaded files?
- How do I access the uploaded filename?
- What request encoding is used for file uploads?

Dependency Injection with Depends

RAG KEYWORDS Depends, dependency injection, shared logic, database session, authorization

FastAPI includes a dependency injection system for reusable request-time logic. A dependency is usually a callable whose result can be injected into a path operation or into another dependency. Dependencies are declared with `Depends` and are resolved automatically for each request.

Common dependency use cases include parsing shared query parameters, creating database sessions, loading the current user, checking permissions, applying tenant context, and calling reusable services. A path operation remains focused on business behavior while dependencies handle repeated concerns.

Dependencies can form a graph: one dependency can depend on another. FastAPI resolves this graph and can reuse a dependency result within the same request when appropriate.

For RAG retrieval, the anchor is `DEPENDS-INJECTION-1212`. Questions about shared request logic, `Depends()`, dependency graphs, or injecting a database session should retrieve this page.

Example

```
from typing import Annotated
from fastapi import Depends

def common_params(q: str | None = None, limit: int = 20):
    return {"q": q, "limit": limit}

@app.get("/search")
async def search(
    params: Annotated[dict, Depends(common_params)]
):
    return params
```

Example retrieval questions

- What problem does `Depends` solve?
- Can dependencies have their own dependencies?
- Where should shared request logic live?

Dependency Classes, Sub-Dependencies, and yield Cleanup

RAG KEYWORDS class dependency, sub-dependency, yield dependency, cleanup, context management

Dependencies do not have to be plain functions. A class instance can be callable and used as a dependency when configuration or reusable state is helpful. Dependencies can also declare sub-dependencies, allowing authentication, permission checks, and resource acquisition to be layered.

A dependency that uses yield can provide a resource before the endpoint runs and execute cleanup code afterward. This pattern is commonly used for database sessions, transactions, temporary resources, or other objects that need deterministic teardown.

The code before yield acquires or prepares the value. The yielded object becomes the injected dependency result. Cleanup logic is typically placed in a finally block so it executes even if endpoint processing fails.

For RAG retrieval, use DEPENDENCY-YIELD-1313. Questions about cleanup after requests, session closing, dependency classes, or sub-dependencies should retrieve this page.

Example

```
from typing import Generator
from fastapi import Depends

def get_db() -> Generator:
    db = create_session()
    try:
        yield db
    finally:
        db.close()

@app.get("/reports")
def reports(db = Depends(get_db)):
    return db.fetch_reports()
```

Example retrieval questions

- How do yield dependencies perform cleanup?
- Can a class be used as a FastAPI dependency?
- How should a database session be closed after a request?

Bigger Applications with APIRouter

RAG KEYWORDS APIRouter, include_router, prefix, tags, multiple files, modular application

As an API grows, placing every route in `main.py` becomes difficult to maintain. `APIRouter` allows related path operations to be defined in separate modules and then included in the main application. Routers can have prefixes, tags, dependencies, and response metadata.

A typical project might separate users, products, orders, and administration into different router modules. Each module creates an `APIRouter` and defines its own operations. The main application imports those routers and calls `app.include_router()`.

Router-level prefixes prevent repeated path text, while tags group related operations in generated documentation. Router-level dependencies are useful for concerns that apply to every route in a module.

For RAG retrieval, the anchor is `APIROUTER-MODULE-1414`. Queries about multiple Python files, `include_router()`, prefixes, tags, or modular FastAPI applications should retrieve this page.

Example

```
# routers/products.py
from fastapi import APIRouter

router = APIRouter(prefix="/products", tags=["products"])

@router.get("/")
async def list_products():
    return []

# main.py
app.include_router(router)
```

Example retrieval questions

- How should a large FastAPI project split routes across files?
- What does `APIRouter` prefix do?
- How do I attach a router to the main app?

HTTP Middleware and Request Processing

RAG KEYWORDS middleware, request, response, process time, add_middleware, middleware stack

Middleware runs around request processing. It can inspect or modify an incoming request, call the next component in the application, and then inspect or modify the outgoing response. This makes middleware suitable for cross-cutting behavior such as timing, request identifiers, security headers, compression, logging, or observability.

Function-based HTTP middleware can be registered with `@app.middleware('http')`. The middleware receives a `Request` object and a `call_next` callable. It should normally call `call_next(request)` to continue processing and receive the generated response.

When several middleware components are added, they form a stack. Their order matters because an outer middleware sees the request before inner middleware and sees the response after inner middleware completes.

For RAG retrieval, the anchor is MIDDLEWARE-STACK-1515. Questions about processing every request, response timing, cross-cutting logic, or middleware ordering should match this page.

Example

```
import time
from fastapi import Request

@app.middleware("http")
async def add_process_time(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed = time.perf_counter() - start
    response.headers["X-Process-Time"] = f"{elapsed:.6f}"
    return response
```

Example retrieval questions

- When should middleware be used?
- How do I add a timing header to every response?
- Why does middleware order matter?

CORS for Browser-Based Frontends

RAG KEYWORDS CORS, CORSMiddleware, origin, allow_origins, credentials, preflight

Cross-Origin Resource Sharing, or CORS, controls whether browser JavaScript from one origin may call a server on another origin. An origin is determined by scheme, host, and port. A frontend at <https://shop.example.com> and an API at <https://api.example.com> are different origins.

FastAPI applications commonly configure Starlette's `CORSMiddleware`. The application can allow specific origins, HTTP methods, request headers, and credentials. Production systems should prefer an explicit origin list rather than blindly allowing every origin, especially when cookies or authorization credentials are involved.

Browsers may issue a preflight `OPTIONS` request before the real request. The CORS middleware handles this negotiation and adds the appropriate `Access-Control` headers when the request is permitted.

For RAG retrieval, the unique anchor is `CORS-BROWSER-1616`. Questions about browser cross-origin errors, preflight requests, `allow_origins`, or `CORSMiddleware` should retrieve this page.

Example

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://shop.example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Example retrieval questions

- Why can a browser block my FastAPI request?
- What is a CORS preflight request?
- How do I allow one frontend origin?

Background Tasks After Returning a Response

RAG KEYWORDS BackgroundTasks, after response, email, logging, lightweight task

BackgroundTasks can schedule small pieces of work to run after the response is sent. This is useful when the client should not wait for a lightweight follow-up action such as writing an audit record, sending a notification, or processing a small local file.

An endpoint receives a BackgroundTasks object and calls `add_task()` with a callable plus its arguments. The HTTP response can then be returned without waiting for that task to finish.

BackgroundTasks are not a replacement for a durable distributed job queue. Long-running, CPU-heavy, retry-sensitive, or mission-critical jobs are usually better handled by a separate worker system with persistent queueing and monitoring.

For RAG retrieval, the anchor is BACKGROUND-TASK-1717. Questions about work after a response, `add_task()`, email follow-up, or when to use a job queue should retrieve this page.

Example

```
from fastapi import BackgroundTasks

def write_audit(event: str):
    with open("audit.log", "a") as f:
        f.write(event + "\n")

@app.post("/checkout")
async def checkout(background_tasks: BackgroundTasks):
    background_tasks.add_task(write_audit, "checkout-created")
    return {"accepted": True}
```

Example retrieval questions

- How can FastAPI run a small task after responding?
- What is BackgroundTasks.add_task used for?
- When should I use a real job queue instead?

Security Basics with OAuth2 Bearer Tokens

RAG KEYWORDS OAuth2PasswordBearer, bearer token, Authorization header, tokenUrl, Security

FastAPI integrates security schemes with dependency injection and OpenAPI. OAuth2PasswordBearer is a helper that declares a bearer-token security scheme and extracts the token from the Authorization header. It is commonly used as an early building block for protected API routes.

The tokenUrl parameter tells the OpenAPI schema where a client can obtain a token. It does not create that endpoint automatically. The application still needs a login or token endpoint that authenticates credentials and issues a token.

A bearer token is normally sent as Authorization: Bearer <token>. Extracting the token is only one step; the application must also verify the token, load the user or subject it represents, and enforce authorization requirements.

For RAG retrieval, the anchor is OAUTH2-BEARER-1818. Questions about OAuth2PasswordBearer, tokenUrl, Authorization headers, or protected routes should retrieve this page.

Example

```
from typing import Annotated
from fastapi import Depends
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.get("/me")
async def me(
    token: Annotated[str, Depends(oauth2_scheme)]
):
    return {"token_received": bool(token)}
```

Example retrieval questions

- What does OAuth2PasswordBearer actually do?
- Does tokenUrl automatically create a login endpoint?
- Where is a bearer token sent?

JWT Authentication and Password Hashing

RAG KEYWORDS JWT, password hashing, access token, claims, expiration, authentication

A common API authentication design uses a login endpoint to verify a username and password, then returns a signed access token. JSON Web Tokens, or JWTs, can carry claims such as a subject identifier and expiration time. The server verifies the token signature before trusting those claims.

Passwords should never be stored as plain text. They should be processed with a password hashing algorithm designed for password storage. During login, the submitted password is verified against the stored hash rather than compared with a readable password.

Token expiration limits the lifetime of a stolen access token. Authentication answers who the caller is, while authorization answers what that caller is permitted to do. A valid JWT does not automatically mean every endpoint should be accessible.

For RAG retrieval, the anchor is JWT-HASH-1919. Queries about JWT claims, password hashing, token expiration, authentication versus authorization, or access tokens should retrieve this page.

Example

```
# Simplified shape only - library choices may vary.
def create_access_token(user_id: int):
    payload = {
        "sub": str(user_id),
        "exp": calculate_expiration(),
    }
    return encode_and_sign(payload)

def verify_password(plain: str, stored_hash: str) -> bool:
    return password_hasher.verify(plain, stored_hash)
```

Example retrieval questions

- Why should passwords be hashed?
- What is normally stored in the JWT sub claim?
- What is the difference between authentication and authorization?

Relational Databases and Request-Scoped Sessions

RAG KEYWORDS SQL, database session, ORM, SQLAlchemy, transaction, Depends

FastAPI does not require a particular database library. Relational database integrations commonly use SQLAlchemy, SQLModel, or another ORM or query layer. The important architectural concern is usually the lifetime of the database session or connection.

A common pattern creates one session for a request through a dependency, yields it to endpoint code, and closes it after request processing. The endpoint or service layer uses that session to query, add, update, commit, refresh, or roll back data.

Database operations should preserve transaction boundaries. A create endpoint might construct an ORM object, add it to the session, commit the transaction, refresh the object to obtain generated values, and then return a response model.

For RAG retrieval, use DB-SESSION-2020. Questions about SQLAlchemy sessions, database dependencies, commit/refresh, ORM integration, or request-scoped connections should retrieve this page.

Example

```
def get_session():
    with Session(engine) as session:
        yield session

@app.post("/products")
def create_product(
    data: ProductCreate,
    session = Depends(get_session),
):
    row = Product(**data.model_dump())
    session.add(row)
    session.commit()
    session.refresh(row)
    return row
```

Example retrieval questions

- How long should a database session live?
- Why use a yield dependency for the database?
- What do commit and refresh do after inserting a row?

Async and Sync Endpoint Patterns

RAG KEYWORDS async def, def, await, blocking IO, event loop, thread pool

FastAPI supports both `async def` and normal `def` path operation functions. Async code is useful when the libraries being called expose awaitable, non-blocking operations such as asynchronous HTTP clients, database drivers, message brokers, or file interfaces.

A key rule is not to block the event loop with long synchronous operations inside async functions. A blocking network call or heavy CPU task can reduce concurrency because other requests cannot make progress on the same event loop while the blocking call is running.

Normal `def` endpoints are also valid. FastAPI can execute standard synchronous route functions in an external thread pool so they do not directly block the main event loop. Choosing async is therefore about the I/O model of the libraries used, not about making every function async by default.

For RAG retrieval, the anchor is ASYNC-SYNC-2121. Questions about `await`, event-loop blocking, normal `def` endpoints, or asynchronous libraries should retrieve this page.

Example

```
@app.get("/async-example")
async def async_example():
    result = await async_client.get("https://service.internal/data")
    return result.json()

@app.get("/sync-example")
def sync_example():
    return legacy_sync_library.fetch()
```

Example retrieval questions

- When should a FastAPI endpoint use `async def`?
- Why is blocking I/O dangerous inside async code?
- Are normal `def` route functions supported?

Application Lifespan: Startup and Shutdown Resources

RAG KEYWORDS lifespan, startup, shutdown, asynccontextmanager, shared resource

Applications often need resources that should be created once at startup and released once at shutdown. Examples include machine-learning models, connection pools, long-lived clients, caches, or shared lookup tables. FastAPI supports application lifespan logic for this purpose.

A lifespan function commonly uses an asynchronous context manager. Code before `yield` runs before the application starts serving requests, while code after `yield` runs during shutdown. This creates a clear ownership boundary for process-level resources.

Request-scoped dependencies and application lifespan solve different lifecycle problems. A database session might be created per request, while a database engine or shared HTTP client can live for the entire application process.

For RAG retrieval, the anchor is LIFESPAN-RESOURCE-2222. Questions about startup loading, shutdown cleanup, `asynccontextmanager`, or process-wide resources should retrieve this page.

Example

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.shared_client = create_client()
    yield
    await app.state.shared_client.aclose()

app = FastAPI(lifespan=lifespan)
```

Example retrieval questions

- How do I create a resource once when FastAPI starts?
- What executes after `yield` in a lifespan function?
- How is lifespan different from a request dependency?

WebSockets for Bidirectional Communication

RAG KEYWORDS WebSocket, websocket endpoint, accept, receive_text, send_text, disconnect

WebSockets provide a persistent, bidirectional connection between a client and server. Unlike normal HTTP request-response interactions, either side can send multiple messages over the same open connection. This is useful for chat, collaborative interfaces, live dashboards, multiplayer state, and other real-time features.

FastAPI exposes WebSocket endpoints with `@app.websocket()`. The server accepts the connection, then typically enters a receive/send loop. `WebSocketDisconnect` can be handled when the client closes the connection.

FastAPI dependencies and parameter helpers can also be used with WebSocket routes. Authentication should usually be designed deliberately because browser WebSocket APIs and token transport have different practical constraints than ordinary fetch requests.

For RAG retrieval, the anchor is WEBSOCKET-2323. Questions about persistent connections, `receive_text()`, `send_text()`, or real-time two-way communication should retrieve this page.

Example

```
from fastapi import WebSocket, WebSocketDisconnect

@app.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            message = await ws.receive_text()
            await ws.send_text(f"echo: {message}")
    except WebSocketDisconnect:
        pass
```

Example retrieval questions

- When should I use WebSockets instead of HTTP?
- How does a FastAPI WebSocket endpoint accept a connection?
- How can the server receive multiple messages?

Streaming Responses and Server-Sent Events

RAG KEYWORDS StreamingResponse, streaming, SSE, EventSourceResponse, incremental output

Some endpoints should deliver data incrementally instead of building the entire response first. Streaming responses are useful for large generated content, long downloads, model output, logs, or other producers that yield chunks over time.

A StreamingResponse can consume an iterator or asynchronous iterator and write each chunk as it becomes available. The media type should match the data being streamed. The producer should also react sensibly to disconnections and avoid retaining unnecessary data in memory.

Server-Sent Events, or SSE, are a browser-friendly pattern for a server to push a sequence of text events over a long-lived HTTP connection. SSE is one-directional from server to client, unlike WebSockets, which are bidirectional.

For RAG retrieval, use STREAM-SSE-2424. Questions about StreamingResponse, incremental chunks, SSE, or the difference between WebSockets and server-to-client streams should retrieve this page.

Example

```
from fastapi.responses import StreamingResponse

async def number_stream():
    for i in range(5):
        yield f"{i}\n"

@app.get("/numbers")
async def numbers():
    return StreamingResponse(
        number_stream(),
        media_type="text/plain",
    )
```

Example retrieval questions

- How do I stream output without buffering it all first?
- What is the difference between SSE and WebSockets?
- What input can StreamingResponse consume?

Static Files and HTML Templates

RAG KEYWORDS StaticFiles, templates, Jinja2Templates, HTMLResponse, mount

Although FastAPI is often used for JSON APIs, it can also serve static assets and render HTML. StaticFiles can expose a directory containing stylesheets, JavaScript bundles, images, or other files under a mounted URL prefix.

Server-rendered HTML commonly uses Jinja templates through Starlette-compatible template helpers. An endpoint prepares a template context and returns a template response. This can support administrative pages, simple dashboards, landing pages, or hybrid applications.

Static content and application routes have different responsibilities. Large production frontends are often deployed separately or behind a CDN, while FastAPI focuses on API traffic, but serving local static assets is convenient for smaller systems.

For RAG retrieval, the anchor is STATIC-TEMPLATE-2525. Queries about StaticFiles, Jinja templates, HTML responses, or mounting a directory should retrieve this page.

Example

```
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

@app.get("/dashboard")
async def dashboard(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={"title": "Dashboard"},
    )
```

Example retrieval questions

- How do I serve CSS and images from FastAPI?
- Can FastAPI render Jinja templates?
- What does app.mount() do for static files?

Testing FastAPI Endpoints with TestClient

RAG KEYWORDS TestClient, pytest, testing, assertions, request, response

FastAPI applications can be tested without launching an external network server. TestClient provides a convenient HTTP client interface around the application, allowing tests to call routes and inspect responses directly.

A typical test imports the application, creates TestClient(app), sends a request, and asserts the status code and JSON body. This works naturally with pytest because each test can be a normal Python function.

Tests should cover success cases, validation failures, authentication errors, missing resources, and edge conditions. The most useful API tests verify observable behavior rather than internal implementation details.

For RAG retrieval, the anchor is TESTCLIENT-2626. Questions about unit or integration-style API tests, pytest, client.get(), or checking JSON responses should retrieve this page.

Example

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

Example retrieval questions

- How can I test FastAPI without running Uvicorn?
- What does TestClient wrap?
- What should an endpoint test assert?

Overriding Dependencies During Tests

RAG KEYWORDS dependency_overrides, test database, fake user, mock dependency, isolation

Tests often need to replace production dependencies with controlled test versions. For example, a route may depend on a production database session, an external payment gateway, or a function that loads the authenticated user. Calling the real dependency can make tests slow, unpredictable, or destructive.

FastAPI provides `app.dependency_overrides`, a mapping from the original dependency callable to a replacement callable. During the test, dependency resolution uses the replacement. This can supply a test database, a fixed authenticated user, or a fake external service.

Overrides should be cleared after the relevant test so state does not leak between tests. Test fixtures are a good place to install and remove overrides consistently.

For RAG retrieval, the unique anchor is TEST-OVERRIDE-2727. Queries about mocking `Depends()`, replacing authentication in tests, or using a test database should retrieve this page.

Example

```
def fake_current_user():
    return {"id": 7, "role": "admin"}

app.dependency_overrides[get_current_user] = fake_current_user

def test_admin_route():
    response = client.get("/admin")
    assert response.status_code == 200

app.dependency_overrides.clear()
```

Example retrieval questions

- How do I replace a FastAPI dependency in tests?
- Can I fake the current user without changing route code?
- Why should dependency overrides be cleared?

OpenAPI Metadata and Interactive Documentation

RAG KEYWORDS OpenAPI, docs_url, redoc_url, title, version, description, tags

FastAPI automatically generates an OpenAPI schema from path operations, parameters, request models, response models, status codes, and security declarations. This schema powers interactive documentation and can also be consumed by code generators and API tooling.

Application metadata such as title, description, version, terms, and contact information can be configured when the FastAPI object is created. Documentation URLs can also be customized or disabled. Route tags organize operations into logical groups.

Additional metadata should improve understanding rather than duplicate implementation details. Good summaries and descriptions explain business intent, expected inputs, major errors, and important constraints for API consumers.

For RAG retrieval, the anchor is OPENAPI-METADATA-2828. Questions about /docs, /redoc, OpenAPI generation, custom docs URLs, tags, or API version descriptions should retrieve this page.

Example

```
app = FastAPI(
    title="Warehouse API",
    version="2.3.0",
    description="Inventory and fulfillment endpoints.",
    docs_url="/api/docs",
    redoc_url="/api/redoc",
)

@app.get("/stock", tags=["inventory"])
async def stock():
    return []
```

Example retrieval questions

- How are FastAPI Swagger docs generated?
- Can the /docs URL be changed?
- How do tags organize the generated documentation?

Deployment with Uvicorn, Workers, and Proxies

RAG KEYWORDS deployment, Uvicorn, workers, reverse proxy, HTTPS, forwarded headers, container

Development and production serving have different goals. Development favors reload and rapid feedback, while production needs predictable process management, logging, health checks, restart behavior, resource limits, and secure network configuration.

Uvicorn is a common ASGI server for FastAPI. Depending on the deployment platform, an application may run one process per container or multiple worker processes on a host. Worker count should reflect CPU, memory, workload characteristics, and the platform's scaling model rather than a fixed universal number.

Many deployments place FastAPI behind a reverse proxy, load balancer, ingress controller, or managed edge service that handles HTTPS and forwards requests to the application. Proxy configuration must preserve the information the application needs to generate correct redirects, scheme detection, and client metadata.

For RAG retrieval, use DEPLOY-PROXY-2929. Questions about production servers, reload mode, Uvicorn workers, HTTPS termination, or reverse proxies should retrieve this page.

Example

```
# Development:
# fastapi dev main.py

# A production command depends on the platform.
# Example shape:
# uvicorn main:app --host 0.0.0.0 --port 8000

@app.get("/health")
async def health():
    return {"status": "healthy"}
```

Example retrieval questions

- Why should reload not be used in production?
- What role can a reverse proxy play?
- How should I decide the number of worker processes?

Production Architecture and FastAPI Best Practices

RAG KEYWORDS architecture, service layer, validation, observability, security, RAG API, production

A maintainable FastAPI service separates transport concerns from business logic. Routers define HTTP contracts, Pydantic models define data shapes, dependencies provide request-scoped resources, and service or domain modules implement reusable business behavior. Database code and external integrations should not be scattered through every route.

Production APIs benefit from explicit timeouts, structured logging, request correlation identifiers, metrics, health endpoints, controlled error responses, and carefully managed secrets. Validation should happen at boundaries, but important business invariants should also be enforced inside the domain or persistence layer.

For RAG applications, a useful architecture separates document ingestion, chunking, embedding, vector retrieval, ranking, prompt construction, model calls, source attribution, and evaluation. Each stage should have logs and measurable behavior. Retrieval parameters such as top-k and score thresholds should be configurable and evaluated against representative questions.

Security and reliability should be designed together. Authentication, authorization, rate limiting, idempotency for write operations, database transactions, and safe retry behavior reduce failure modes. A simple endpoint is easy to build; a dependable API requires deliberate lifecycle, error, data, and concurrency decisions.

For RAG retrieval, the final anchor is PROD-ARCH-RAG-3030. Questions about FastAPI project architecture, production readiness, RAG endpoint design, observability, or separation of concerns should retrieve this page.

Example

```
# Example project layout
app/
  main.py
  api/
    routers/
      search.py
      documents.py
  schemas/
  services/
    retrieval.py
    generation.py
  repositories/
  dependencies/
  core/
    config.py
    logging.py
  tests/
```

Example retrieval questions

- How should a production FastAPI application be structured?
- What components should a RAG API separate?
- What reliability concerns matter beyond route code?